

Chapter 9. Linting with Ruff and pre-commit

In 1978, Stephen C. Johnson, a researcher at Bell Labs, wrote a program that could detect a number of bugs and obscurities in C code. He named the program after the fluff on your pullover when you take it out of the washing machine: Lint. It was to become the first in a long line of *linters*, programs that analyze source code and point out problematic constructs.

Linters don't *run* a program to discover issues with it; they read and analyze its source code. This process is known as *static analysis*, as opposed to *runtime* (or *dynamic*) *analysis*. It makes linters both fast and safe—you needn't worry about side effects, such as requests to production systems. Static checks can be smart and also fairly complete—you needn't hit the right combination of edge cases to dig up a latent bug.

NOTE

Static analysis is powerful, but you should still write tests for your programs. Where static checks use deduction, tests use observation. Linters verify a limited set of generic code properties, while tests can validate that a program satisfies its requirements.

Linters are also great at enforcing a readable and consistent style, with a preference for idiomatic and modern constructs over obscure and deprecated syntax. Organizations have adopted style guides for years, such as the recommendations in [PEP 8](#) or the [Google Style Guide for Python](#). Linters can function as *executable* style guides: by flagging offending constructs automatically, they keep code review focused on the meaning of a change rather than stylistic nitpicks.

This chapter has three parts:

- The first part introduces the Ruff linter, a Rust implementation of Python linters that automatically fixes many of the issues it detects.
- The second part describes pre-commit, a linter framework with Git integration.
- The third part presents the Ruff code formatter, a Rust implementation of the Black code style.

But first, let's look at a typical problem that linters help you solve.

Linting Basics

The constructs flagged by linters may not be outright illegal. More often, they just trigger your spider-sense that something might be wrong.

Consider the Python code in [Example 9-1](#).

Example 9-1. Can you spot the problem?

```
import subprocess

def run(command, args=[], force=False):
    if force:
        args.insert(0, "--force")
    subprocess.run([command, *args])
```

If you haven't been bitten by this bug before, you may be surprised to find that the function sometimes passes `--force` to the command when it shouldn't:

```
>>> subprocess.run = print # print commands instead of running them
>>> run("myscript.py", force=True)
['myscript.py', '--force']
>>> run("myscript.py")
['myscript.py', '--force']
```

The bug is known as a *mutable argument default*. Python evaluates argument defaults when a function is defined, not when you call it. In other

words, both of your calls used the same list as the default for `args`. The first call appended the item `--force`, so that item got passed to the second call as well.

Linters can detect pitfalls like this, warn you about them, and even fix them for you. Let's use a linter named Ruff on the function—you'll hear a lot more about it in this chapter. For now, just note its error message, which identifies the bug:¹

```
$ pipx run ruff check --extend-select B
bad.py:3:23: B006 Do not use mutable data structures for argument defaults
Found 1 error.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option)
```

Two tools have dominated linting for much of Python’s history: Pylint and Flake8.

[Pylint](#) (2001) is the *grande dame* of Python linting. It takes an all-encompassing approach, with hundreds of checks. It requires a certain amount of upfront configuration, but it’s famous for its thoroughness.

In 2010, a simpler tool with an out-of-the-box experience saw the light of day: [Flake8](#). It doesn’t define any checks of its own—instead, it exposes a plugin system for other linters. It runs two fundamental linters by default: Pyflakes detects syntax errors and potential bugs, while pycodestyle enforces the PEP 8 style guide. (A third linter, McCabe, limits code complexity; it’s disabled by default.) Flake8 laid the groundwork for a vast and growing ecosystem of linter plugins.

In recent years, the Python code quality ecosystem has seen significant changes, spearheaded by three tools—Black, mypy, and Ruff:

- The rise of the code formatter [Black](#) has made countless stylistic checks unnecessary. It’s named after Henry Ford’s adage, “any color the customer wants, as long as it’s black.” Black uncompromisingly applies the same code style to every project that adopts it.
- Type checkers like [mypy](#) have taken over the large realm of type-related linter checks—for example, detecting function calls with the wrong type of argument. I’ll discuss type checking in [Chapter 10](#).
- [Ruff](#) reimplements an entire ecosystem of Python code-quality tooling in Rust, including Flake8, Pylint, and Black. Besides offering a unified experience, Ruff speeds up checks by up to two orders of magnitude.²

Historically, linters only emitted warnings about offending code, leaving the thankless task of improving it to humans. Modern linters can fix many violations automatically, including sophisticated tasks such as refactoring a Python codebase to use modern language features.

The Ruff Linter

Ruff is a blazingly fast open source Python linter and code formatter, written in the Rust programming language. Ruff’s linter reimplements dozens of Python linters—including many Flake8 plugins, Pylint, pyupgrade, Bandit, isort, and more.

Astral, the company behind Ruff, also created the Python packaging tool uv (see [“Managing Environments with uv”](#)), and it has assumed the stewardship of Rye, a Python project manager (see [“Managing Packages with Rye”](#)). All of these tools are implemented in Rust.

TIP

If you manage your project with Rye, the Ruff linter and code formatter are available under the commands `rye lint` and `rye fmt`, respectively.

Install Ruff globally with pipx:

```
$ pipx install ruff
```

But wait—how can pipx install a Rust program? The Ruff binary is available as a wheel on PyPI, so Python folks like you and me can install it with good old pip and pipx. You could even run it with `py -m ruff`.

Let’s look at another example of working with Ruff. Consider this refactoring of HTTP headers, which replaces a list with a dictionary:

```
headers = [f"User-Agent: {USER_AGENT}"] # version 1
headers = {f"User-Agent": USER_AGENT}   # version 2
```

When you refactor f-strings, it’s easy to leave the `f` prefix behind after removing placeholders. Ruff flags f-strings without placeholders—they’re noisy, they confuse readers, and somebody might have forgotten to include a placeholder.

Run the command `ruff check`—the front-end for Ruff’s linter. Without arguments, the command lints every Python file under your current directory, unless it’s listed in a `.gitignore` file:

```
$ ruff check
example.py:2:12: F541 [*] f-string without any placeholders
Found 1 error.
[*] 1 fixable with the `--fix` option.
```

[Figure 9-1](#) takes a closer look at Ruff’s diagnostic message.

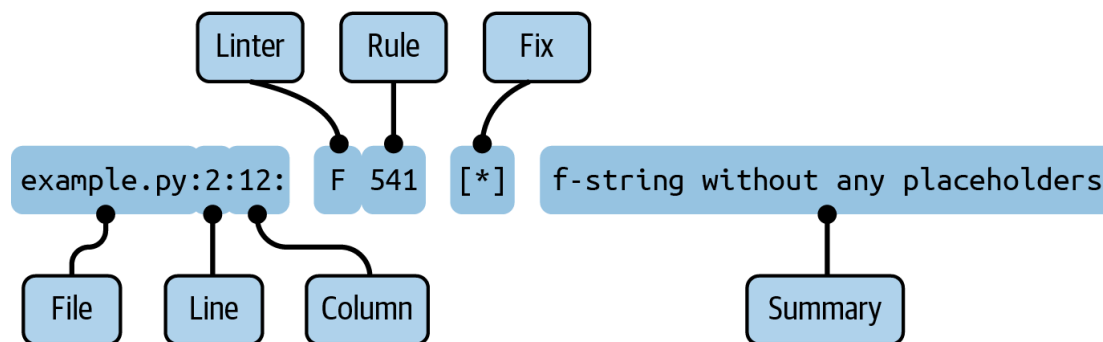


Figure 9-1. A diagnostic message from Ruff

Ruff tells you where the violation occurred—the file, line, and line offset—and gives you a short summary of what went wrong: *f-string without any placeholders*. Two interesting bits are sandwiched between the location and the summary: an alphanumeric code (`F541`) identifies the linter rule, and the sign `[*]` indicates that Ruff can automatically fix the issue.

If you’re ever confused why you’re getting a warning, you can ask Ruff to explain it using the command `ruff rule`:

```
$ ruff rule F541
f-string-missing-placeholders (F541)

Derived from the Pyflakes linter.

Fix is always available.

What it does
Checks for f-strings that do not contain any placeholder expressions.

Why is this bad?
```

f-strings are a convenient way to format strings, but they are not necessary if there are no placeholder expressions to format.
...

Rule codes have a prefix of one or more letters, followed by three or more digits. The prefix identifies a specific linter—for example, the `F` in `F541` stands for the Pyflakes linter. Ruff reimplements many more Python code-quality tools—as of this writing, it ships over 50 built-in plugins modeled after existing tools. You can find out which linters are available using the command `ruff linter`:

```
$ ruff linter
  F Pyflakes
E/W pycodestyle
C90 mccabe
  I isort
  N pep8-naming
  D pydocstyle
  UP pyupgrade
... (50+ more lines)
```

You can activate linters and individual rules for your project in its `pyproject.toml` file. The setting `tool.ruff.lint.select` enables any rules whose code starts with one of the given prefixes. Out of the box, Ruff enables some basic all-around checks from Pyflakes and pycodestyle:

```
[tool.ruff.lint]
select = ["E4", "E7", "E9", "F"]
```

Pyflakes and pycodestyle

Pyflakes (`F`) flags constructs that are almost certainly mistakes, such as unused imports or the pointless f-strings you’ve seen above. It steers clear of any questions of style. Pycodestyle (whose rules use the prefixes `E` and `W`), detects violations of PEP 8, the style guide originally written by Python’s inventor, Guido van Rossum, together with Barry Warsaw and Alyssa Coghlan.

Ruff enables only a subset of pycodestyle by default, because code formatters have obviated many of its checks. Still, PEP 8 makes style recommendations that go beyond mere code formatting. Would you agree that `x is not None` sounds more natural than `not x is None`? The default rules detect and fix a number of such issues, leading to code that's easier to read and understand.

TIP

If you aren't using an opinionated code formatter, consider enabling the entire `E` and `W` blocks. Their automatic fixes help ensure minimal PEP 8 compliance. They're similar to, but not yet as feature-complete as, the `autopep8` formatter (see [“Approaches to Code Formatting: autopep8”](#)).³

Fantastic Linters and Where to Find Them

Ruff has too many rules to describe in this book, and more are being added all the time. How do you find the good ones for your project? Try them out! Depending on your project, you may want to enable individual rules (`"B006"`), groups of rules (`"E4"`), entire plugins (`"B"`), or even every existing plugin at the same time (`"ALL"`).

WARNING

Reserve the special `ALL` code for experimentation: it will implicitly enable new linters whenever you upgrade Ruff. *Beware*: some plugins require configuration to produce useful results, and some rules conflict with other rules.⁴

Besides `select`, Ruff has an `extend-select` directive that selects rules in addition to the default set (see [“Linting Basics”](#)). Generally, you should prefer the `select` directive, because it keeps your configuration self-contained and explicit:

```
[tool.ruff.lint]
select = ["E", "W", "F", "B006"]
```


If you’re unsure where to start, [Table 9-1](#) describes a dozen built-in plugins to try.

Table 9-1. A dozen widely useful Ruff plugins

Prefix	Name	Description
RUF	Ruff-specific rules	A collection of lints native to Ruff
I	isort	Group and sort import statements
UP	pyupgrade	Use modern language features compatible with your target Python version
SIM	flake8-simplify	Use idiomatic constructs to simplify code
FURB	refurb	Use idiomatic constructs to make good code even better
PIE	flake8-pie	A collection of miscellaneous lints
PERF	Perflint	Avoid performance antipatterns
C4	flake8-comprehensions	Use list, set, and dict comprehensions
B	flake8-bugbear	Eliminate likely bugs and design problems
PL	Pylint	A large collection of rules from the mother of all Python linters
D	pydocstyle	Require well-formed docstrings for your functions, classes, and modules
S	flake8-bandit	Detect potential security vulnerabilities

When onboarding legacy projects to Ruff, your first task will be to decide which linters provide the most useful feedback. At this stage, individual diagnostics can be quite overwhelming. It helps to zoom out using the `--statistics` option:

```
$ ruff check --statistics --select ALL
123      I001      [*] Import block is un-sorted or un-formatted
```

```
45     ARG001 [ ] Unused function argument: `bindings`
39     UP007 [*] Use `X | Y` for type annotations
32     TRY003 [ ] Avoid specifying long messages outside the exception class
28     SIM117 [ ] Use a single `with` statement with multiple contexts
23     SLF001 [ ] Private member accessed: `_blob`
17     FBT001 [ ] Boolean-typed positional argument in function definition
10     PLR0913 [ ] Too many arguments in function definition (6 > 5)
...
```

At this point, you have two options. First, if a linter is particularly noisy, hide it from the output using the `--ignore` option. For example, if you're not ready to add type annotations and docstrings, exclude `flake8-annotations` and `pydocstyle` with `--ignore ANN,D`. Second, if you see a linter with interesting findings, enable it permanently in *pyproject.toml* and fix its warnings. Rinse and repeat.

TIP

Work toward enforcing the same set of linters for all your projects, with the same configurations, and prefer default configurations over customizations. This will make your codebase more consistent and accessible across the entire organization.

Disabling Rules and Warnings

The `select` setting is flexible but purely additive: it lets you opt into rules whose code starts with a given prefix. The `ignore` setting lets you fine-tune in the other direction: it disables individual rules and rule groups. Like `select`, it matches rule codes by their prefixes.

The subtractive method is handy when you need most, but not all, of a linter's rules, and when you're adopting a linter gradually. The `pydocstyle` plugin (`D`) checks that every module, class, and function has a well-formed docstring. Your project may be *almost* there, with the exception of module docstrings (`D100`). Use the `ignore` setting to disable all warnings about missing module docstrings until you've fully onboarded your project:

```
[tool.ruff.lint]
select = ["D", "E", "F"]
ignore = ["D100"] # Don't require module docstrings for now.
```

The `per-file-ignore` setting lets you disable rules for a part of your codebase. Here's another example: the `bandit` plugin (5) has a rich inventory of checks to help you detect security vulnerabilities in your code. Its rule `S101` flags every use of the `assert` keyword.⁵ But you still need `assert` to express expectations in `pytest` (see [Chapter 6](#)). If your test suite lives in a `tests` directory, disable `S101` for its files like this:

```
[tool.ruff.lint.per-file-ignores]
"tests/*" = ["S101"] # Tests can use assertions.
```

Disabling rules should be a last resort. It's usually better to suppress individual warnings by adding a special comment to offending lines. This comment has the form `# noqa:` followed by one or more rule codes.

WARNING

Always include rule codes in your `noqa` comments. Blanket `noqa` comments can hide unrelated issues. Marking violations also makes them easier to find when you're ready to fix them. Use the rule `PGH004` from the `pygrep-hooks` linter to require rule codes.

The `noqa` system lets you silence false positives as well as legitimate warnings that you choose not to prioritize at this time. For example, the MD5 message-digest algorithm is generally agreed to be insecure, and Bandit's `S324` flags its uses. But if your code interacts with a legacy system that requires you to compute an MD5 hash, you may not have a choice. Disable the warning with a `noqa` comment:

```
md5 = hashlib.md5(text.encode()).hexdigest() # noqa: S324
```

Bandit's checks often flag constructs that deserve close scrutiny, without meaning to outright ban them. The idea is that you will vet the offending

lines one by one and suppress the warning if you determine that the specific usage is innocuous.

It can be reasonable to enable a rule and suppress *all* of its warnings. This lets you enforce a rule going forward only—that is, only when you touch a region of code. Ruff supports this workflow with the `--add-noqa` option, which inserts `noqa` comments to all offending lines on your behalf:

```
$ ruff check --add-noqa
```

Like every comment, `noqa` comments can become outdated—for example, a refactoring may have inadvertently fixed the suppressed warning. Stale `noqa` comments are noisy and create friction when you're out to squash linter violations. Luckily, Ruff is in an excellent position to remediate this. Its rule `RUF100` automatically removes `noqa` comments that no longer apply.

Automation with Nox

Without corrective measures, the code quality of large projects degrades over time. Linting, as an automated, mandatory check, helps to counteract that natural entropy. Nox (see [Chapter 8](#)) is an automation framework that lets you run linters as a part of your mandatory checks.

Here's a Nox session that runs Ruff on every Python file in the current directory:

```
@nox.session
def lint(session):
    session.install("ruff")
    session.run("ruff", "check")
```

Nox is a valid choice here, but when it comes to linting, there's a more convenient and powerful alternative: pre-commit, a cross-language linter framework with Git integration.

The pre-commit Framework

Pre-commit is a tool and framework that lets you add third-party linters to a project with minimal boilerplate. Linters in various languages come with ready-to-use integrations for pre-commit, called *hooks*. You can run these hooks explicitly from the command line, or you can configure your local repository to run them whenever you commit changes (and on some other events).

Install pre-commit globally using pipx:

```
$ pipx install pre-commit
```

First Steps with pre-commit

Let's add a pre-commit hook for Ruff to your project. Create a file named *.pre-commit-config.yaml* in the top-level directory, with contents as in [Example 9-2](#). You'll find a short YAML fragment like this in the public documentation of most linters that support pre-commit.

Example 9-2. A *.pre-commit-config.yaml* file with a hook for Ruff

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.4.4
  hooks:
  - id: ruff
```

Authors distribute their pre-commit hooks via Git repositories. In the *.pre-commit-config.yaml* file, you specify the URL, revision, and hooks for each repository you want to use. The URL can be any location Git can clone from. The revision is most commonly a Git tag pointing to the latest release of the linter. A repository can have more than one hook—for example, Ruff provides `ruff` and `ruff-format` hooks for its linter and code formatter, respectively.

Pre-commit is intimately tied to Git, and you must invoke it from within a Git repository. Let's establish a baseline by linting every file in the repository, using the command `pre-commit run` with the `--all-files` option:

```
$ pre-commit run --all-files
[INFO] Initializing environment for https://github.com/astral-sh/ruff-pre-commit
[INFO] Installing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
ruff.....Passed
```

When you run a hook for the first time, pre-commit clones the hook repository and installs the linter into an isolated environment. This can take some time, but you don't have to do it often: pre-commit caches the linter environments across multiple projects.

A Hook Up Close

If you're curious how a pre-commit hook works under the hood, take a peek at Ruff's [hook repository](#). The file `.pre-commit-hooks.yaml` in the repository defines the hooks. [Example 9-3](#) shows an excerpt from the file.

Example 9-3. An excerpt from the `.pre-commit-hooks.yaml` file for Ruff

```
- id: ruff
  name: ruff
  language: python
  entry: ruff check --force-exclude
  args: []
  types_or: [python, pyi]
```

Every hook comes with a unique identifier and a friendly name (`id` and `name`). Refer to hooks by their unique identifier when you interact with pre-commit. Their names appear only in console messages from the tool.

The hook definition tells pre-commit how to install and run the linter by specifying its implementation language (`language`) and its command and command-line arguments (`entry` and `args`). The Ruff hook is a Python

package, so it specifies Python as the language. The `--force-exclude` option ensures that you can exclude files from linting. It tells Ruff to honor its `exclude` setting even when pre-commit passes excluded source files explicitly.

TIP

You can override the `args` key in your `.pre-commit-config.yaml` file to pass custom command-line options to a hook. By contrast, command-line arguments in the `entry` key are mandatory—you can't override them.

Finally, the hook declares which file types the linter understands (`types_or`). The `python` file type matches files with `.py` or related extensions and executable scripts with a Python shebang. The `pyi` file type refers to stub files with type annotations (see [“Distributing Types with Python Packages”](#)).

For a Python hook, pre-commit creates a virtual environment in its cache directory. It installs the hook by running, essentially, `pip install .` inside the hook repository. When it's time to run the hook, pre-commit activates the virtual environment and invokes the command with any selected source files.

[Figure 9-2](#) shows a developer machine with three Python projects using pre-commit hooks. Pre-commit clones the hook repositories into its cache directory and installs the hooks into isolated environments. Hook repositories define hooks in `.pre-commit-hooks.yaml` files, while projects reference the hooks in `.pre-commit-config.yaml` files.

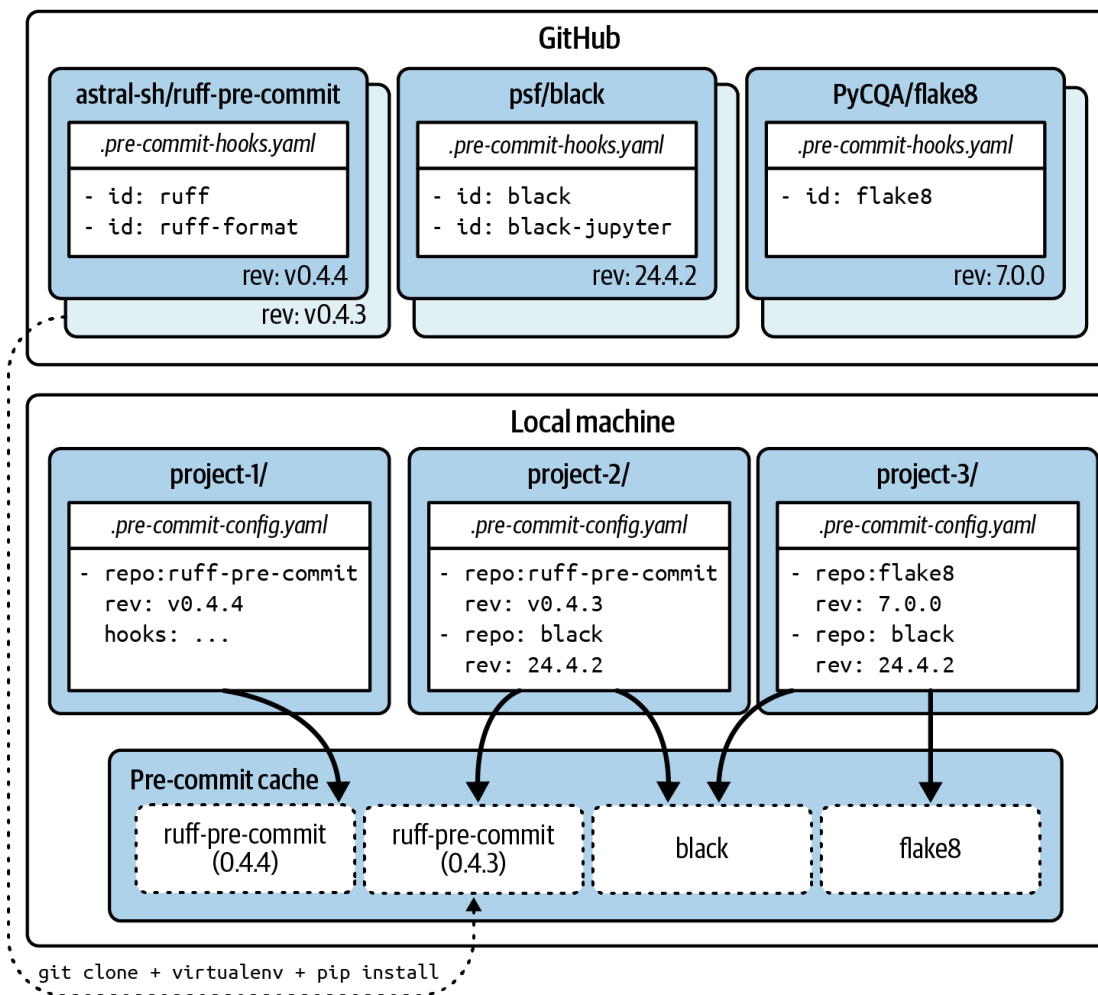


Figure 9-2. Three projects with pre-commit hooks for Ruff, Black, and Flake8.

Automatic Fixes

Modern linters can fix many violations by modifying the offending source files in place. Linters with automatic fixes eliminate entire classes of bugs and code smells⁶ at nearly zero cost. Like code formatters, they have caused a paradigm shift in software development, letting you focus on higher-level concerns without compromising on code quality.

By convention, most pre-commit hooks enable automatic fixes by default. Thanks to Git, they can apply fixes with relative safety, without risk of overwriting your work irreversibly. That said, they work best if you commit early and often.

WARNING

Automatic fixes bring tremendous benefits, but they assume some basic Git hygiene: don't pile up uncommitted changes in your repository (or stash them before linting). Pre-commit saves and restores your local modifications in some contexts, but not all.

Let's try this out. When Ruff detects the mutable argument default, it indicates that you can enable a “hidden” fix. (Ruff asks you to opt into the fix because people might conceivably depend on mutable defaults, say, for caching.) First, enable the linter rule and the fix in *pyproject.toml*:

```
[tool.ruff.lint]
extend-select = ["B006"]
extend-safe-fixes = ["B006"]
```

Ruff's pre-commit hook requires you to opt in with the `--fix` option, as shown in [Example 9-4](#). The options `--show-fixes` and `--exit-non-zero-on-fix` ensure that all violations are displayed in the terminal and result in a nonzero exit status, even if Ruff was able to fix them.

Example 9-4. Enabling automatic fixes for the Ruff hook

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.4.4
  hooks:
  - id: ruff
    args: ["--fix", "--show-fixes", "--exit-non-zero-on-fix"]
```

Save [Example 9-1](#) in a file called *bad.py*, commit the file, and run pre-commit:

```
$ pre-commit run --all-files
ruff.....Failed
- hook id: ruff
- exit code: 1
- files were modified by this hook
```

```
Fixed 1 error:
- bad.py:
    1 x B006 (mutable-argument-default)

Found 1 error (1 fixed, 0 remaining).
```

If you inspect the modified file, you'll see that Ruff has replaced the argument default with `None`. The empty list is now assigned inside the function, giving every call its own instance of `args`:

```
def run(command, args=None, force=False):
    if args is None:
        args = []
    if force:
        args.insert(0, "--force")
    subprocess.run([command, *args])
```

Instead of inspecting the modified files, you can also run `git diff` to see the changes applied to your code. Alternatively, you can tell pre-commit to show you a diff of the fixes right away, using the option `--show-diff-on-fail`.

Running pre-commit from Nox

Pre-commit gives you access to production-ready integrations for linters in many languages. For this reason alone, I recommend running linters from pre-commit instead of Nox. (If you still need convincing, see the next section, which presents another compelling reason to use pre-commit.)

That said, you still should include a Nox session for pre-commit itself. This ensures that you can run all the checks for your project with a single command, `nox`. [Example 9-5](#) shows how to define the session. If your `noxfile.py` sets `nox.options.sessions`, add the session to that list, as well.

Example 9-5. A Nox session for linting with pre-commit

```
nox.options.sessions = ["lint", "tests"]
```

```
@nox.session
def lint(session):
    options = ["--all-files", "--show-diff-on-fail"]
    session.install("pre-commit")
    session.run("pre-commit", "run", *options, *session.posargs)
```

By default, pre-commit runs every hook you’ve configured for your project. You can run specific hooks by passing them as additional command-line arguments. This comes in handy when addressing complaints from a specific linter. Thanks to `session.posargs` (see [“Session Arguments”](#)), this also works from Nox:

```
$ nox --session=lint -- ruff
```

Having a single entry point to checks and tasks, including linters, greatly reduces friction for everybody working on a project. But you shouldn’t stop there. Pre-commit was designed to be triggered from Git on every commit. The next section explains how to set up projects to lint changes as you commit them (and why it’s great).

Running pre-commit from Git

Running linters on each commit is a game changer, for three reasons:

- You remove the overhead and distraction of invoking checks manually. Linters run in the background, alerting you only if a violation is found.
- You run checks as early as possible. As a general rule, the earlier you spot an issue, the cheaper it is to fix. (Say goodbye to CI failures due to stylistic nitpicks.)
- It’s fast: you only lint files staged for a commit, instead of the entire codebase.⁷

Set up Git to invoke pre-commit on every commit by running the following command inside your project:

```
$ pre-commit install
pre-commit installed at .git/hooks/pre-commit
```

This command installs a short wrapper script into the `.git/hooks` directory that transfers control to `pre-commit` (Figure 9-3). Programs in the `.git/hooks` directory are known as *Git hooks*. When you run `git commit`, Git invokes the *pre-commit* Git hook. The hook, in turn, invokes `pre-commit`, which runs Ruff and any other pre-commit hooks you have.

Git hooks let you trigger actions at predefined points during Git's execution. For example, the *pre-commit* and *post-commit* Git hooks run before and after Git creates a commit. You've probably guessed by now which of these Git hooks `pre-commit` installs by default—but it supports the other Git hooks as well, if you need them.

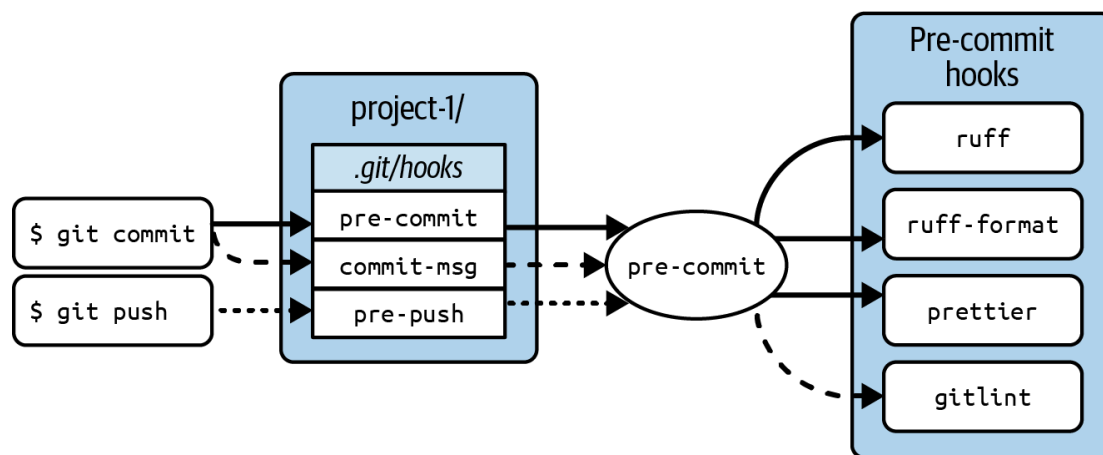


Figure 9-3. Git hooks and pre-commit hooks

Most pre-commit hooks plug into the *pre-commit* Git hook—but not all. Linters for commit messages like `commitlint` and `gitlint` use the *commit-msg* Git hook. Git calls this hook after you’ve composed a commit message. You can install it using the `--hook-type` or `-t` option:

```
$ pre-commit install -t commit-msg
pre-commit installed at .git/hooks/commit-msg
```

But passing this option is easy to forget. If you use Git hooks other than *pre-commit*, list them in the *.pre-commit-config.yaml* file instead:

```
default_install_hook_types: [pre-commit, commit-msg]
```

Place this directive at the top level of your pre-commit configuration.

[Figure 9-4](#) depicts a typical workflow with pre-commit. On the left, there’s a file you’re editing in your project (*worktree*); the center represents the staging area for the next commit (*index*); and the current commit is on the right (*HEAD*).

Initially, the three areas are in sync. Suppose you remove the placeholder from the f-string, but forget to remove the `f` prefix from the string literal (marked as ❶ in [Figure 9-4](#)). You stage your edit using `git add` (❷) and run `git commit` to create a commit (❸). Before your editor can pop up for the commit message, Git transfers control to pre-commit. Ruff promptly catches your mistake and fixes the string literal in your worktree (❹).

At this point, all three areas have different contents. Your worktree contains your change with Ruff’s fix, the staging area has your change without the fix, and *HEAD* still points to the commit before your change. This lets you audit the fix by comparing the worktree to the staging area, using `git diff`. If you’re happy with what you see, you can stage the fix with `git add` (❺) and retry the commit with `git commit` (❻).

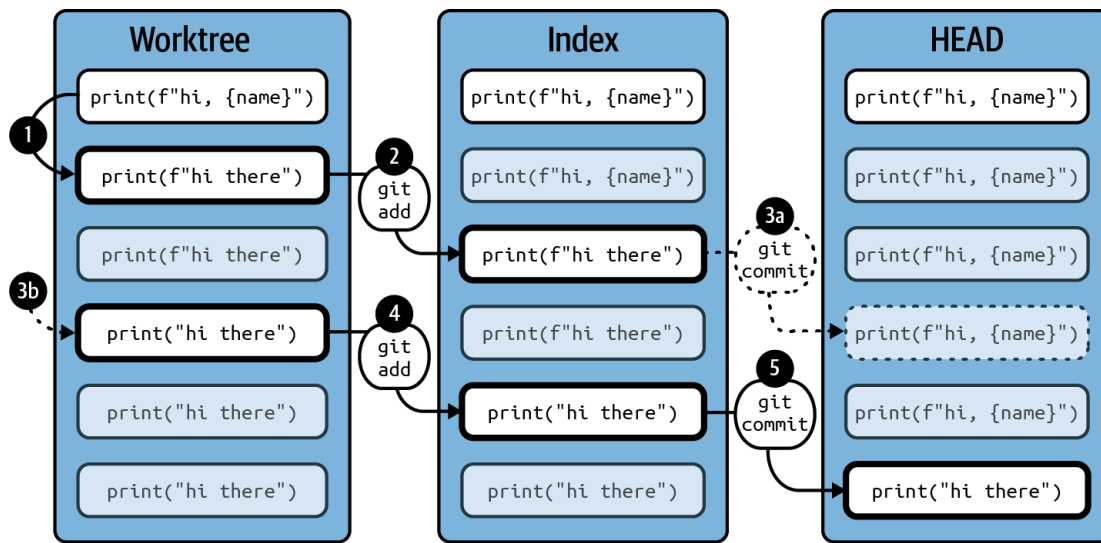


Figure 9-4. Workflow with pre-commit

With automatic fixes, this workflow reduces the interference of linters to a minimum, rerunning the commit. But sometimes you don't want to be distracted by linters at all—for example, you may want to record some work in progress. Git and pre-commit give you two options to get your commit past a stubborn linter. First, you can skip Git hooks entirely using the `--no-verify` or `-n` option:

```
$ git commit -n
```

Alternatively, you can skip a specific pre-commit hook using the `SKIP` environment variable (which also takes a comma-separated list, if you need to skip more than one hook):

```
$ SKIP=ruff git commit
```

Git hooks control which changes enter your local repository, but they're voluntary—they don't replace CI checks as a gatekeeper for the default branch in your shared repository. If you already run Nox in CI, the session in [Example 9-5](#) takes care of that.

Skipping hooks doesn't help with false positives or when you want to deploy a critical fix despite minor nits: your mandatory checks would still fail in CI. In these cases, you'll need to advise the linter to ignore the specific violation (see [“Disabling Rules and Warnings”](#)).

The Ruff Formatter

Over the course of months, Ruff reimplemented a plethora of Python linters behind the `ruff check` command and saw wide adoption in the Python world. A bit over a year in, Ruff acquired the `ruff format` command.⁸ The Ruff formatter reimplements the de facto standard of Python code formatting, Black, in Rust. It provides yet another building block for the integrated and highly performant toolchain that Ruff has become for Python.

But let's start at the beginning.

Many years ago, I was working in a small team on a C++ codebase. Our processes were simple: everybody committed directly to the main branch. There was no CI pipeline—just nightly builds and a dashboard showing compiler errors, warnings, and test failures. Every now and then, one of us went over recent changes with refactorings and style cleanups. It may sound surprising, but the codebase was in good shape. Nothing beats mutual understanding in a close-knit team.

A few years down the road, cracks appeared in our workflow. The company had grown—new engineers were unfamiliar with the coding conventions, senior engineers struggled to communicate them. On top of this, the team had inherited a legacy codebase that didn't adhere to any discernible rules at all.

We started to look at automation. Maybe tooling could take over the common chore of identifying code-quality issues and fixing them across the codebase. In 2013, a code formatter named `clang-format` had been released as part of the LLVM compiler toolkit, and it looked promising.

After a brief evaluation, I decided against it.

By design, `clang-format` completely rewrote each source file according to the configured style. I was worried about its all-or-nothing approach for a few reasons. First, despite dozens of configuration settings, it would erase many of our style conventions. Second, it would be hard to review the resulting large patch. Third, since reformatting would touch a large part of the codebase, it would become hard to establish the historical context of a line of code using the version control system.

In hindsight, I was wrong—all of these concerns can be addressed.

The evolution of code formatters in Python eventually changed my mind—specifically the meteoric rise of Black. But before addressing the concerns above, let's take a step back and look at the situation in Python as it was when Black entered the scene.

Approaches to Code Formatting: autopep8

Consider the following gem of Python code:

```
def create_frobicator_factory(the_factory_name,
                              interval_in_secs=100,  dbg=False,
                              use_singleton=None, frobnicate_factor=4.5):
    if dbg: print('creating frobnication factory '+the_factory_name+"...")
    if (use_singleton):  return _frob_sngltn      #we're done
    return FrobnicationFactory( the_factory_name,

                                intrvl = interval_in_secs          ,f=frobnicate_factor    )
```

The function showcases various formatting issues, such as inconsistent alignment, lack or excess of whitespace, and single-line `if` constructs. Some of these clearly hurt readability. Others, such as two-space indentation, deviate from a widely accepted practice codified in PEP 8.

A minimal approach for dealing with stylistic issues of this sort is `autopep8`, an early pioneer of automatic linter fixes and still in use. Building on `pycodestyle`, it surgically corrects offenses while preserving the code layout otherwise.

Let's run `autopep8` on the code example with default settings:

```
$ pipx run autopep8 example.py
```

Here's the function as formatted by `autopep8`:

```
def create_frobicator_factory(the_factory_name,
                              interval_in_secs=100,  dbg=False,
                              use_singleton=None, frobnicate_factor=4.5):
    if dbg:
        print('creating frobnication factory '+the_factory_name+"...")
    if (use_singleton):
        return _frob_sngltn  # we're done
    return FrobnicationFactory(the_factory_name,

                                intrvl=interval_in_secs, f=frobnicate_factor)
```

You'll likely find this easier on the eye. For better or worse, `autopep8` didn't touch some other questionable stylistic choices, such as the rogue blank line in the `return` statement and the inconsistent quote characters. `Autopep8` uses `pycodestyle` to detect issues, and `pycodestyle` had no complaint here.

TIP

Unlike most code formatters, `autopep8` lets you apply selected fixes by passing `--select` with appropriate rule codes. For example, you can run `autopep8 --select=E111` to enforce four-space indentation.

Approaches to Code Formatting: YAPF

Developed at Google in 2015, the YAPF formatter borrows its design and sophisticated formatting algorithm from `clang-format`. The name YAPF stands for “Yet Another Python Formatter.”⁹ YAPF reformats a codebase according to a wealth of configuration options.

Run it like this:

```
$ pipx run yapf example.py
```

Here's YAPF's version of the code, using default settings:

```
def create_froblicator_factory(the_factory_name,
                               interval_in_secs=100,
                               dbg=False,
                               use_singleton=None,
                               frobnicate_factor=4.5):
    if dbg: print('creating frobnication factory ' + the_factory_name + "...")
    if (use_singleton): return _frob_sngltn #we're done
    return FrobnicationFactory(the_factory_name,
                               intrvl=interval_in_secs,
                               f=frobnicate_factor)
```

YAPF's formatting rules touch more ground than `autopep8`—for example, it arranges the function parameters consistently and removes the bogus

empty line. YAPF respects existing formatting choices as long as they're compatible with your configuration. For example, it didn't split the single-line `if` statements or eliminate parentheses around `if` conditions.

An Uncompromising Code Formatter

In 2018, a new code formatter named Black entered the scene. Its core principle: minimal configurability!

Let's try Black on the code example:

```
$ pipx run black example.py
```

Black formats the function as follows:

```
def create_frobnicator_factory(
    the_factory_name,
    interval_in_secs=100,
    dbg=False,
    use_singleton=None,
    frobnicate_factor=4.5,
):
    if dbg:
        print("creating frobnication factory " + the_factory_name + "...")
    if use_singleton:
        return _frob_sngltn # we're done
    return FrobnicationFactory(
        the_factory_name, intrvl=interval_in_secs, f=frobnicate_factor
    )
```

Black doesn't fix individual style offenses like `autopep8`, nor does it enforce your style preferences like YAPF. Rather, Black reduces the source code into a canonical form, using a deterministic algorithm—mostly without taking existing formatting into account. In a certain sense, Black makes code style “disappear.” This normalization massively reduces the cognitive overhead of working with Python code.

Black promotes a form of universal readability across the Python ecosystem. Code is read more often than it's being written—a transparent style

helps. At the same time, Black makes code more “writable” too. When you’re contributing to other people’s code, there’s no need to conform to a bespoke and manual code style. Even in a solo project, Black boosts your productivity: if you configure your editor to reformat on save, you can reduce your keystrokes to a minimum while coding.

Black took the Python world by storm, with project after project deciding to “blacken” their source files.

ONBOARDING A CODEBASE TO BLACK

When you onboard a large codebase to an opinionated code formatter like Black, the resulting diff can present a headache.

First, why give up conscious style choices and carefully handcrafted code? You’ll have to make this decision yourself—but consider the following points: What’s the ongoing cost of enforcing your style without the help of automated tools? Do your code reviews focus on the meaning of a change rather than coding style? How long does it take new engineers to get up to speed?

Second, are the changes safe? Black guarantees that the *abstract syntax tree* (AST) of the source code—that is, the parsed representation of the program, as seen by the interpreter—doesn’t change, except for some well-known divergences that preserve semantic equivalence.¹⁰

Third, when you commit the changes, how do you prevent them from cluttering up the output of `git blame`? It turns out that you can configure Git to ignore the commit when annotating files. Store the full 40-character commit hash in a file named `.git-blame-ignore-revs` in the root of the repository. Then run the following command:

```
$ git config blame.ignoreRevsFile .git-blame-ignore-revs
```

Code hosting services like GitHub increasingly support the `.git-blame-ignore-revs` file, as well.

The Black Code Style

Black’s code style becomes invisible once you’ve worked with it for a while. Inevitably, though, some of its choices have led to controversy, even forks. To understand its formatting rules, it helps to look at Black’s goal of producing readable and consistent source code in a predictable and repeatable way.

Take the default of double quotes for string literals, for example. According to the style recommendations of PEP 257 and PEP 8, both docstrings and English text with apostrophes already require double quotes. Choosing double quotes over single quotes therefore results in a more consistent style overall.

Sometimes Black places a lone `):` after the parameters of a function. Nicknamed the “sad face,” it clearly demarcates the function signature from the body without indenting parameters beyond the standard four spaces. This layout also treats parameter lists like any other bracketed constructs, such as tuples, lists, and set literals.

Black also caps lines at 88 characters (one of its few configurable settings). This tradeoff between lateral eye movement and vertical scrolling is based on a data-driven approach, tested on millions of lines of Python code at Meta.

Another goal of Black is to avoid *merge conflicts*, when concurrent changes to the same region of code cannot be combined without human intervention. The trailing comma—placing a comma behind the last item in a sequence—serves this purpose: it lets you insert before and after the last item without conflict.

NOTE

Reducing dependencies between edits helps different people work on the same code. But it also lets you separate or reorder drive-by bugfixes or refactorings and back out tentative commits before submitting your changes for code review.

As noted, Black’s algorithm is deterministic, and existing layouts hardly affect it. A “blackened” file may look as if it was generated straight from the AST.¹¹ In reality, this can’t be the case, though. For one thing, the AST doesn’t include comments, since they don’t affect program execution.

Black takes some cues from the formatted source code besides comments. One example is the blank lines that divide a function body into logical partitions. Likely the most powerful way of affecting Black’s output, however, is the *magic trailing comma*: if a sequence contains a trailing comma, Black splits its elements across multiple lines, even if they would fit on a single line.

Black provides an escape hatch to let you disable formatting for a region of code ([Example 9-6](#)).

Example 9-6. Disabling formatting for a region of code

```
@pytest.mark.parametrize(
    ("value", "expected"),
    [
        ("first test value", "61df19525cf97aa3855b5aeb1b2bcb89"),
        ("another test value", "5768979c48c30998c46fb21a91a5b266"),
        ("and here's another one", "e766977069039d83f01b4e3544a6a54c"),
    ]
) # fmt: skip
def test_frobnicate(value, expected):
    assert expected == frobnicate(value)
```

Hand-formatting can be useful for program data, such as large tables with properly aligned columns.

Formatting Code with Ruff

Code formatters can process millions of lines of code in a batch, or they can run in quick succession when triggered from your editor or on a busy CI server. Being fast was an explicit goal of Black from the start—it ships binary wheels with native code generated by `mypyc`, a compiler for type-annotated Python. While Black is fast, the Ruff formatter further im-

proves performance thirtyfold, thanks to its efficient Rust implementation.

Ruff aims for full compatibility with the Black code style. Unlike Black, Ruff lets you opt into single quotes and indentation using tabs. However, I'd recommend adhering to Black's widely adopted style.

When run without arguments, `ruff format` processes any Python files beneath the current directory. Instead of invoking Ruff manually, add it to your pre-commit hooks, as shown in [Example 9-7](#).

Example 9-7. Running Ruff from pre-commit as a linter and code formatter

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.4.4
  hooks:
    - id: ruff
      args: ["--fix", "--show-fixes", "--exit-non-zero-on-fix"]
    - id: ruff-format
```

The pre-commit hook for the code formatter comes last. This gives it an opportunity to reformat any automatic fixes made by linters.

Summary

In this chapter, you've seen how to improve and preserve the code quality in your projects using linters and code formatters. Ruff is an efficient reimplement of many Python code-quality tools in Rust, including Flake8 and Black. While it's possible to run Ruff and other tools manually, you should automate this process and include it as a mandatory check in CI. One of the best options is pre-commit, a cross-language linter framework with Git integration. Invoke pre-commit from a Nox session to keep a single entry point for your suite of checks.

- 1** The `B` short code activates a group of checks pioneered by `flake8-bugbear`, a plugin for the Flake8 linter.
- 2** Charlie Marsh, [“Python Tooling Could Be Much, Much Faster”](#), August 30, 2022.
- 3** As of this writing, you’ll also need to enable Ruff’s preview mode. Set `tool.ruff.lint.preview` to `true`.
- 4** My reviewer Hynek points out that setting your project to `ALL` can be a great way to learn about antipatterns from experienced people. You can still opt out of a rule after understanding its rationale. Enabling `ALL` requires a bit more work on your side, but it ensures you don’t miss new rules in a Ruff release.
- 5** What’s wrong with assertions? Nothing, but Python skips them when run with `-O` for optimizations—a common way to speed up production environments. So don’t use `assert` to validate untrusted input!
- 6** Kent Beck and Martin Fowler describe *code smells* as “certain structures in the code that suggest—sometimes, scream for—the possibility of refactoring.” Martin Fowler, *Refactoring: Improving the Design of Existing Code*, second edition (Boston: Addison-Wesley, 2019).
- 7** Running pre-commit from Git is the safest way to run linters with automatic fixes: pre-commit saves and restores any changes you haven’t staged, and it rolls back the fixes if they conflict with your changes.
- 8** Charlie Marsh, [“The Ruff Formatter”](#), October 24, 2023.
- 9** Stephen C. Johnson, the author of Lint, also established this infamous naming convention by writing Yacc (Yet Another Compiler-Compiler) in the early 1970s at Bell Labs.
- 10** [“AST Before and After Formatting”](#), *Black documentation*. Last accessed: March 22, 2024.
- 11** You can inspect the AST of a source file with the standard `ast` module, using `py -m ast example.py`.